Dr. Karl Fürlinger
Lehrstuhl für Kommunikationssysteme und Systemprogrammierung

More on Spinlocks



Spinlock Summary so Far

- We have seen several ways in which we can implement busy-wait (spinning) mutual exclusion
  - **lock**()/**unlock**(), or **acquire**()/**release**() interface
  - Either based on **load and store only** (mostly historic and of theoretical relevance)
  - Or based on **read-modify-write operations**

- Practical considerations for choosing a spinlock implementation
  - Spinning on a **central location** can be problematic for performance
  - Scalability and fairness can be achieved with **queue-based locks**
  - The **MCS lock** is an important example for such a queue-based lock that works well in practice

Notes (1)

- Often spinlocks are combined with scheduler-based synchronization
  - Spin for a while, then yield to OS kernel scheduler
  - E.g., the C++ **std::mutex** will most likely be implemented in such a way, but the C++ specification does not prescribe a specific implementation
- The interface extensions and some of the special-case adaptations described in these slides similarly apply to mutexes like std::mutex
- Pure spinlocks are useful when we know that there will be little contention
  - And of course in system code (e.g., OS code that implements the scheduler-based synchronization)

Notes (2)

- Behavior under **contention** is an important consideration for locks and for other concurrent objects as well

- Def.: **Contention** for a lock occurs when multiple threads try to acquire a lock at the same time
  - **Backoff** is a strategy to deal with contention
  - Idea: If you notice that other threads try to acquire the lock at the same time, wait a little before trying again
  - **Exponential Backoff**: Waiting time doubles with each round of waiting (up to a maximum)
  - **Randomness** can be used to avoid waiting in lockstep (all threads waiting for exactly the same amount of time and then retrying at exactly the same time)

Improvements and Extensions for Locks

- Recursive / reentrant locking
  - Allow the same thread to hold the lock multiple times
- "Trylock" and timeouts
  - Acquire the lock, but not at any cost
- Asymmetric locking
  - When not all threads are treated equally
- Locality-aware locking
  - Improve efficiency on NUMA architectures
- Reader-writer locks
  - Allow concurrent access for readers

Reentrant / Recursive Locks (1)

- Allow the same thread to hold the lock “several times”
  - Can be useful when holding the lock is needed in a function that is called in different contexts, including somewhere the lock may already be held

- Any lock can be turned into a reentrant lock
  - We just need to remember the **owner thread** and **acquire count**
  - A thread that already holds the lock can re-acquire it
  - Others have to wait
  - The lock becomes available only after it has been released the same number of times it has been acquired

```
class ReentrantLock {
  Lock l;
  int owner = none;
  int count = 0;
}
```

Reentrant / Recursive Locks (2)

```
void ReentrantLock::acquire() {
  if( owner!=self ) {
    l.acquire();
    owner = self;
  }
  count++;
}
```

```
void ReentrantLock::release() {
  count--;
  if( count==0 ) {
    owner = none;
    l.release();
  }
}
```

- Recursive locks in C++

```
#include <mutex>

std::mutex m;
std::recursive_mutex r;
```

```
// Thread i
m.lock();
m.lock(); // deadlock

r.lock();
r.lock(); // fine
r.unlock();
r.unlock();
```

"Trylock" and Timeouts

- Trylock attempts to acquire the lock and returns a Boolean indicating success
  - Generalization: try to acquire the lock for a specified amount of time
  - Can be useful for deadlock avoidance or in interactive applications

```
class Lock {

  void acquire();
  void release();

  bool try_acquire();

  bool try_acquire_for(duration d);
  bool try_acquire_until(time t);
}
```

Trylock in C++

- The **mutex** and **recursive_mutex** classes also offer the **try_lock**() method

```
#include <mutex>

std::mutex m;
std::recursive_mutex r;
```

```
if( m.try_lock() ) { // lock acquired?
  // yes: critical section
  m.unlock(); // don't forget to unlock
} else  {
  // no: do something else, retry, etc.
}
```

- timed_mutex class supports timeouts

```
#include <mutex>

std::timed_mutex tm;

tm.try_lock_for(const std::chrono::duration& dur);
tm.try_lock_until(const std::chrono::time_point& tp);
```

Using Locks Requires Discipline

- Locks are used to protect shared resources **by convention**
  - We need to remember to **always** use the same lock to protect the same resources
  - Encapsulation (OOD) can help
- Potential **deadlocks** when using more than one lock
  - Several possible solutions, e.g., always acquire locks in the same order
- No direct support for **structured usage**
  - It is easy to forget to unlock a lock

```
std::mutex l;

void foo(string& s) {
  l.lock();
  if( s.size()==0 )
    return; // nothing to do
  // actual work ...
  l.unlock();
}
```

Structured Usage of Locks (1)

- In C++ the recommendation is to not use locks directly
  - **RAII wrappers** are available as an alternative
  - E.g., a **std::lock_guard** object can be instantiated based on an underlying mutex object
  - Constructor acquires lock, destructor releases lock
  - Even when exceptions occur

```
std::mutex mut;

void foo(string& s) {
  std::lock_guard<std::mutex> guard(mut);

  if( s.size()==0 )
    return; // nothing to do

  // actual work ...
} // lock will be released here, no matter what
```

Structured Usage of Locks (2)

- In Java try-finally can be used to achieve a similar effect

```
mutex.lock();
try {
   ... // body
} finally {
   mutex.unlock();
}
```

Asymmetric Locking

- Scenario: A single **preferred thread** should have **fast-path access** to the lock, while all others go through the **slow-path**
  - Useful when data is **almost always** accessed only by the preferred thread but we still want protection for the **rare general case**
  - We can combine a **simple lock** for the fast path with a **general lock** for the slow path

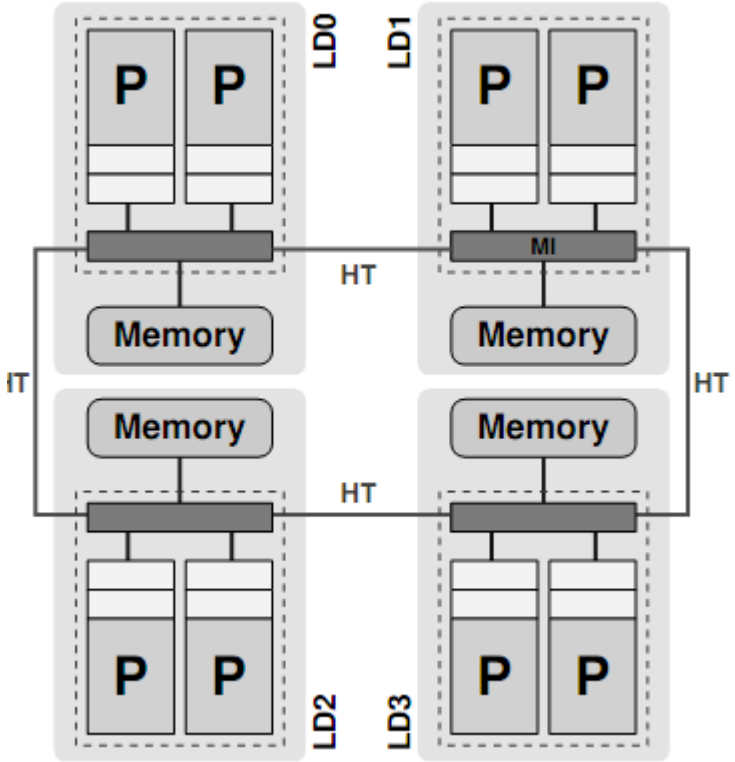
```
class AsymmetricLock {
  PetersonLock p;    // 2-way mutual exclusion
  GeneralLock  l;    // n-way mutual exclusion
}
```

```
acquire():
  if( !preferred(self) )
    g.acquire();
  p.acquire();
```

```
release():
  p.release();
  if( !preferred(self) )
    g.release();
```

Locality-Aware Locking (1)

- Most shared-memory machines are NUMA machines
  - NUMA = Non-Uniform Memory Access
  - Memory access speeds can be different depending on the physical location of the memory



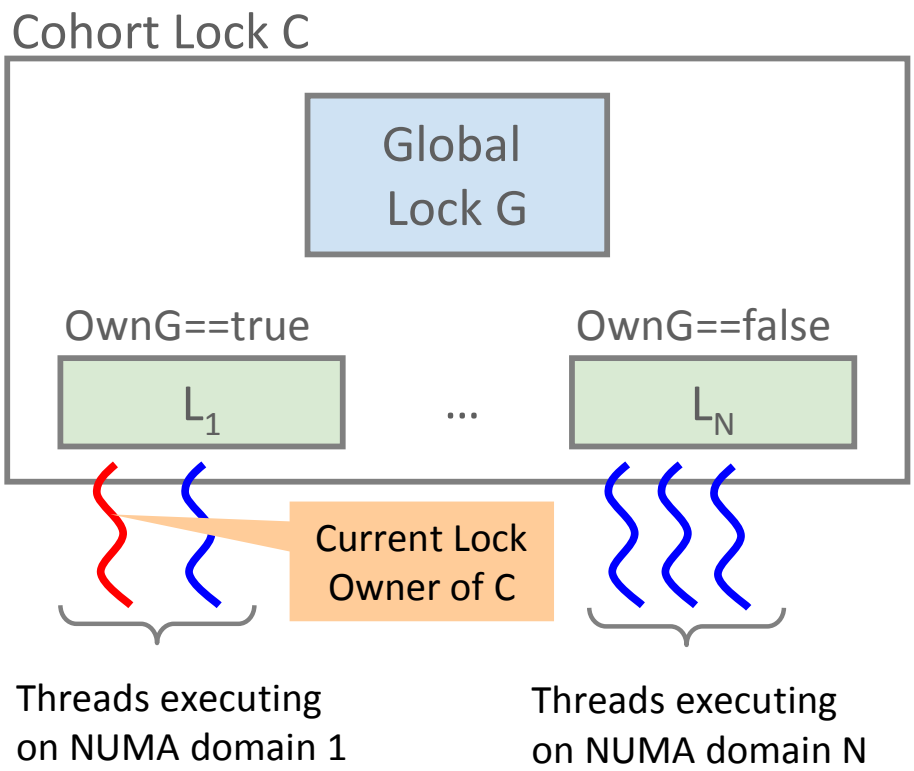
- Example for a NUMA Machine with four NUMA domains (aka. locality domains, LD)

Locality-Aware Locking (2)

- Q: How can we take locality into account for locks?

- A: When multiple threads are waiting to acquire a lock, we can give preference to those in the same NUMA domain as the current lock holder
  - Passing the lock data structure itself will be faster
  - More importantly, the data protected by the lock will also likely be closer to the new thread
  - Of course, there are obvious issues with fairness and starvation to be considered

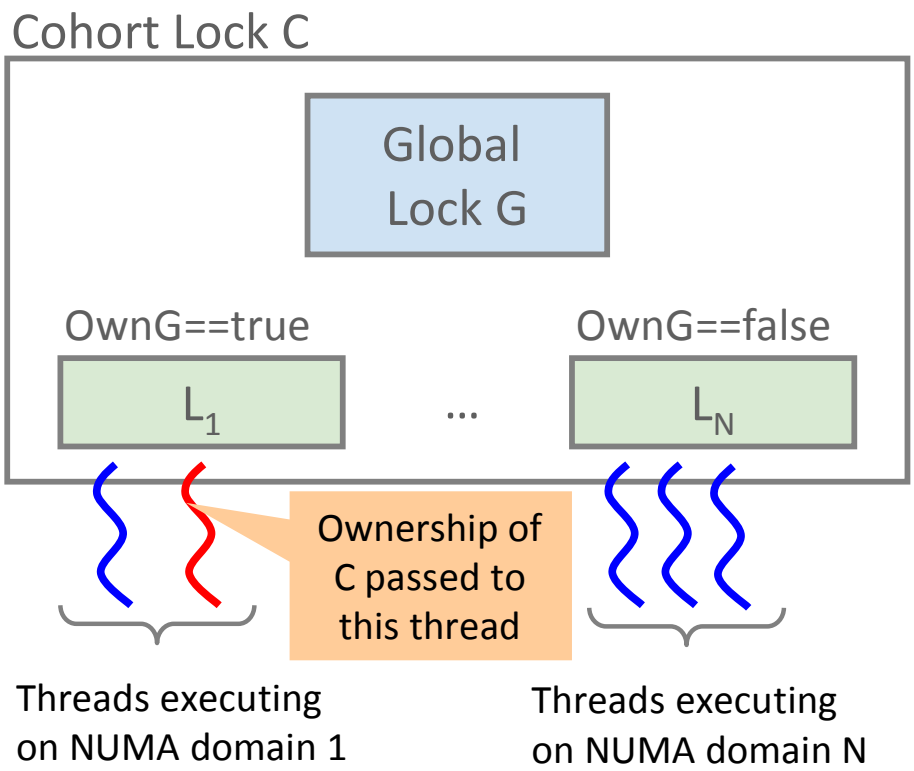
Cohort Locks (1)



- Setup:
  - Global Lock G
  - N local locks L_i (one for each NUMA domain)
  - N flags "OwnG" that indicate if the cohort (set of local threads) owns the global lock
  - Owning the Cohort Lock means **owning a local lock plus owning the global lock**

- Idea:
  - A thread can directly (logically) transfer lock ownership to another thread waiting on the same local lock (the "cohort")
  - The global lock is only released once there are no more local threads, or earlier, if we want to avoid starvation of threads on other NUMA domains

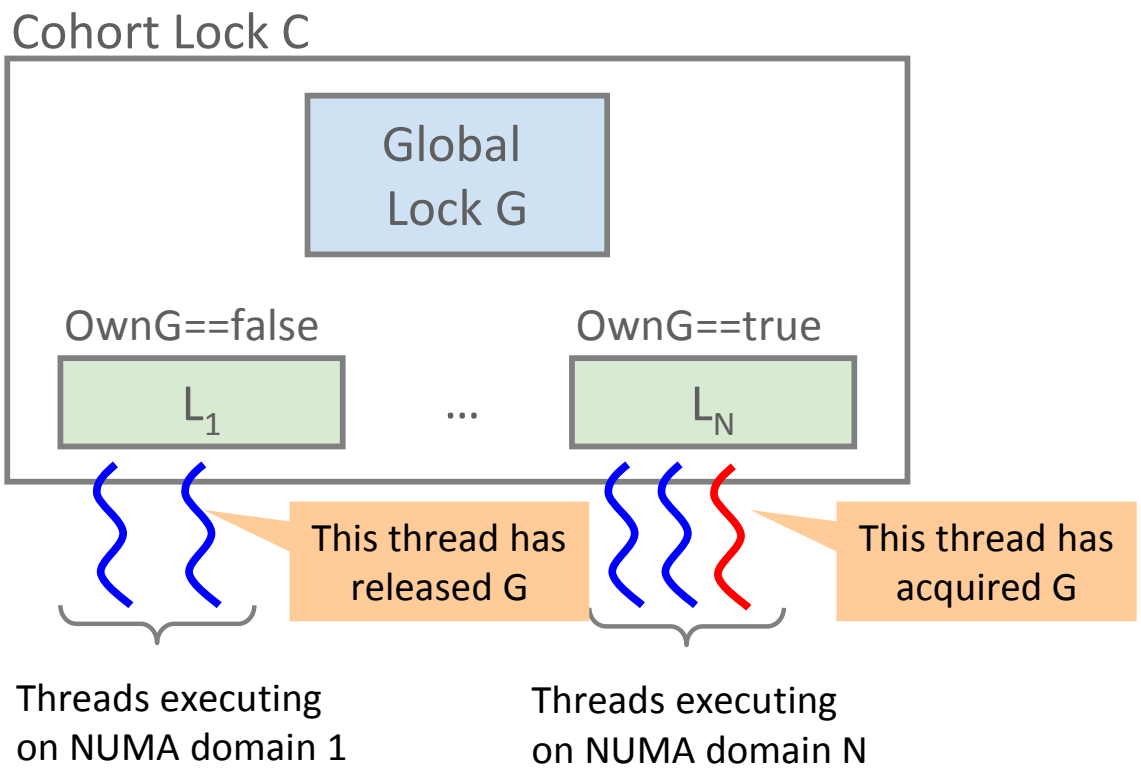
Cohort Locks (2)



- Requirements:
  - G needs to be **thread-oblivious**: G may be released by different thread from the one that acquired it
  - L needs to offer **cohort detection**: When releasing the local lock, we need to be able to tell whether other threads are waiting for the same lock

- As long as these two requirements are met, many combinations are possible
  - E.g., a **TTAS-MCS cohort lock** where G=TTAS, L=MCS

Cohort Locks (3)



- Starvation can be addressed by limiting the number of times a lock may be passed locally before G has to be released
  - Or by limiting the length of time it is held by a numa node

Cohort Locks Pseudo Code

```
class CohortLock {
  lock G;       // global lock
  lock L[];     // local locks, one per NUMA domain
  bool OwnG[]; // which NUMA domain owns G?

acquire():
  id = myNUMA_ID();  // identify my NUMA domain
  L[id].acquire();   // acquire the local lock
  if( !OwnG[id] ) {  // unless we own the global lock
    G.acquire();     // acquire the global lock
    OwnG[id]=true;
  }

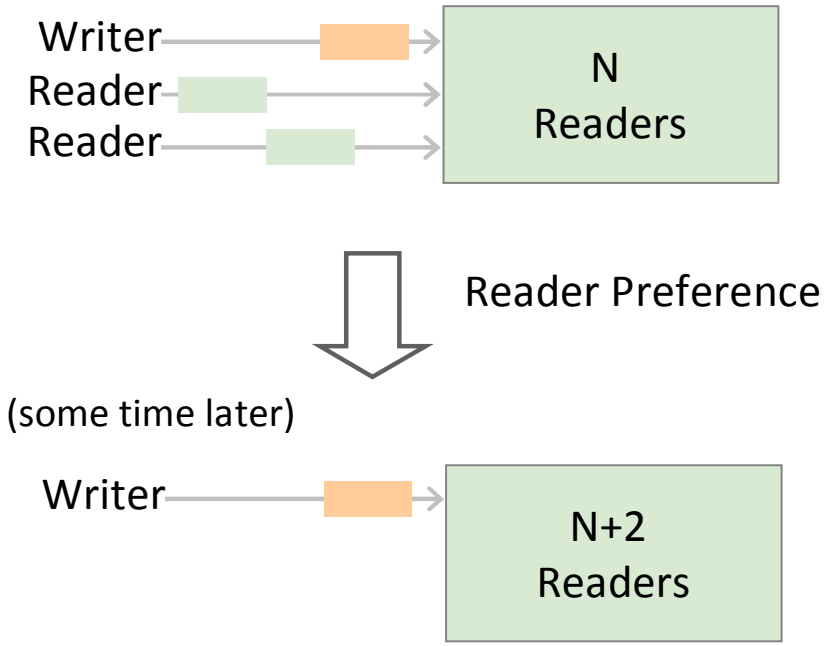
release():
  id = myNUMA_ID();
  if( !cohort_exists() ) { // if no thread waits locally
    G.release();           // release the global lock
    OwnG[id]=false;
  }
  L[id].release();
}
```

Reader-Writer Locks (1)

- Idea: Strict mutual exclusion is **not necessary** if we have threads that just want to read shared data
  - **Multiple readers can be allowed simultaneously**
  - We just need to provide mutual exclusion between readers and any writer and between any two writers

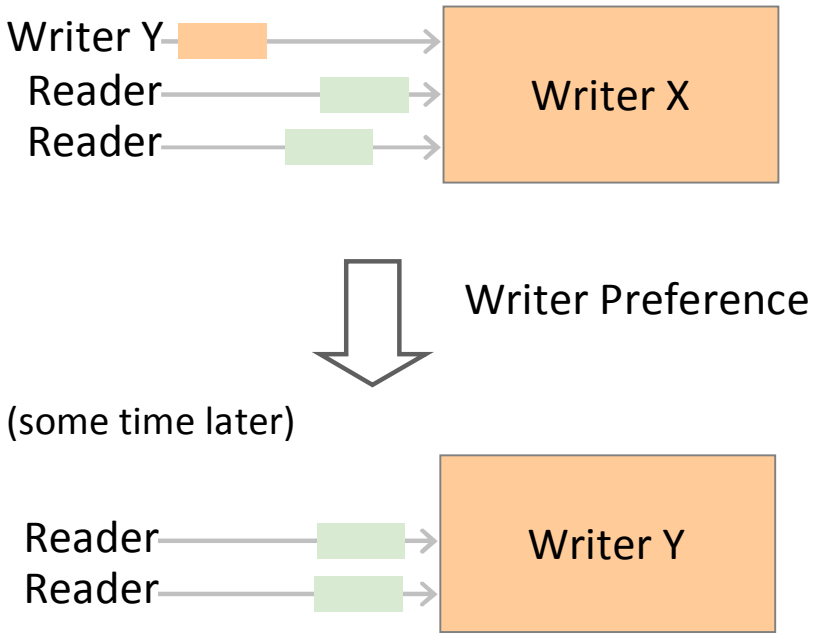
- Several design options exist
  - **Reader preference**: increases total throughput
  - **Writer preference**: makes sure threads see new data quickly
  - **Fair**

RW Locks: Reader Preference



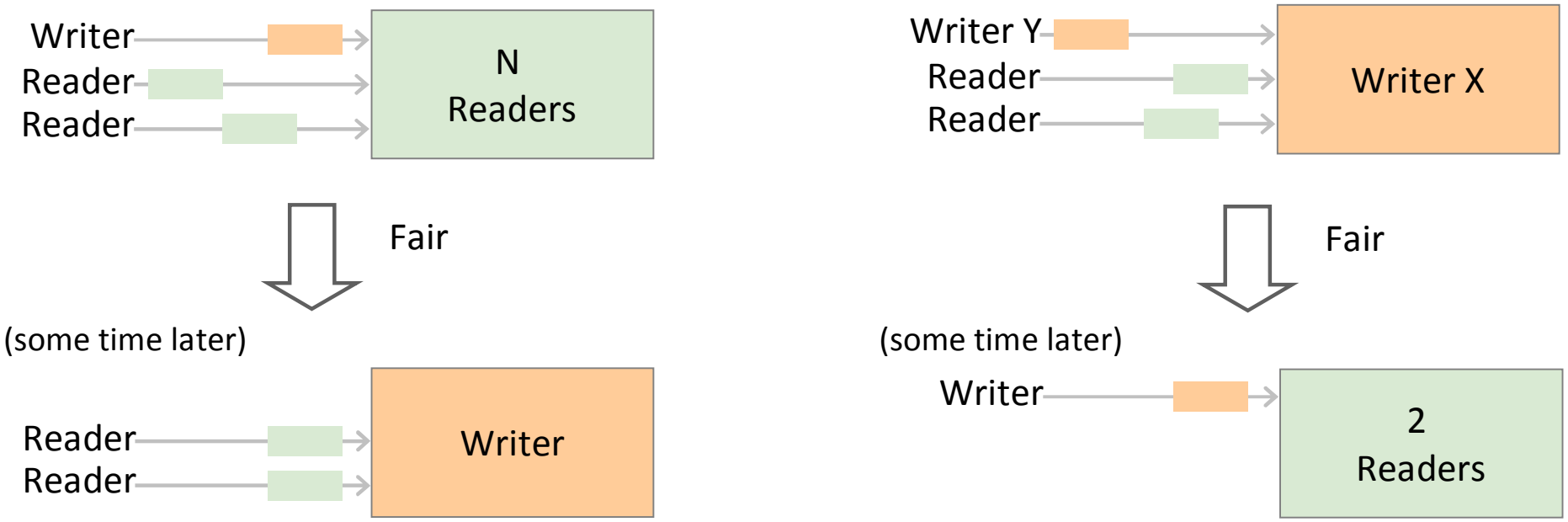
- Reader Preference
  - Readers are allowed to join an existing set of readers, even if a writer is waiting to acquire the lock
  - Waiting time for readers is minimized
  - Increases total throughput of the lock

RW Locks: Writer Preference



- Writer Preference
  - Readers need to wait for current or waiting writers, even if the writer arrived after some of the readers
  - Updates are seen as soon as possible by readers

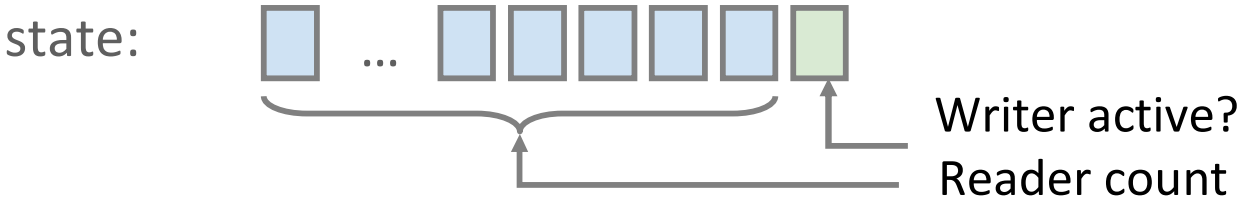
RW Locks: Fair



- Fair RW Locks:
  - Readers wait for earlier writers but join earlier readers
  - Writers wait for earlier readers and writers

A Centralized RW-Lock with Reader Preference

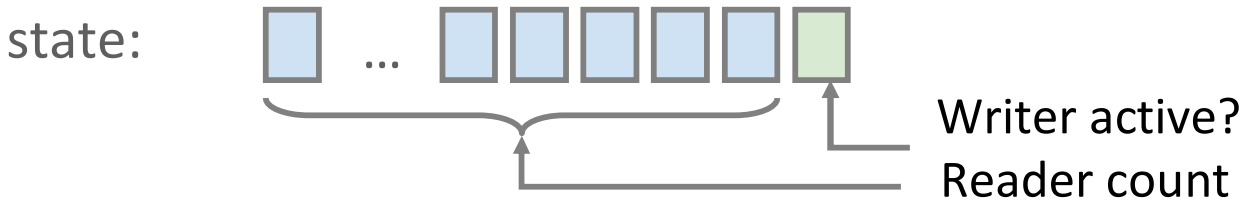
- Idea: Use a **single integer** to represent the state of the lock and use atomic RMW operations for manipulation
  - Lock is either **free** or **held by a writer** or **held by any number of readers**
  - 1 bit for "writer active" and N-1 bits to count the number of readers



0 ... 0 0 0 0 0 0	Lock is free (available)
0 ... 0 0 0 0 0 1	Lock is held by a writer, no readers waiting
0 ... 0 0 0 0 1 1	Lock is held by a writer, one reader waiting
0 ... 0 0 0 1 0 1	Lock is held by a writer, two readers waiting
0 ... 0 0 0 1 0 0	Lock is held by two readers
0 ... 0 0 0 0 1 0	Lock is held by one reader

RW-Lock Interface

- Instead of acquire / release we have two roles:
  - reader_acquire() and reader_release()
  - writer_acquire() and writer_release()



- writer_acquire()
  - wait for empty lock, set "writer active" flag
- writer_release()
  - clear "writer active" flag
- reader_acquire()
  - increment reader count, then spin while "writer active" flag is set
- reader_release()
  - decrement reader count

RW-Lock Pseudo code (1)

```
class RWLock {
  int state = 0;
  const int WA_flag = 1; // writer active
  const int RC_inc = 2;  // increment for reader count
};
```

```
RWLock::writer_acquire() {
  // to acquire the lock as a writer, we need
  // to have no active readers and no active
  // writer => n==0
  while !CAS(&state, 0, WA_flag) { /* spin */ }
  fence(R||RW);
}
```

```
RWLock::writer_release() {
  FAA(&state, -WA_flag, RW||);
}
```

RW-Lock Pseudocode (2)

```
class RWLock {
  int state = 0;
  const int WA_flag = 1; // writer active
  const int RC_inc = 2;  // increment for reader count
};
```

```
RWLock::reader_acquire() {
  FAA(&state, RC_inc)
  while( state.load() & WA_flag ) {} // spin while WA_flag is set
  fence(R||R);
}
```

```
RWLock::reader_release() {
  FAA(&state, -RC_inc, R||);
}
```

Reader-Writer Lock Usecase (1)

- Usecase to demonstrate benefits of RW locks
  - Suppose we have some sort of concurrent **database system** that can be used my multiple threads to execute their transactions
  - The database system offers **read-only** transactions and **read-write** transactions

- A possible simulation of such a database system
  - Simulate a number **N** of threads
  - Each thread performs **M** transactions
  - Each transaction is a write transaction with probability **P**
  - The average duration of a write transaction is $\mathbf{T_{write}}$
  - The average duration of a read transaction is $\mathbf{T_{read}}$
  - The average duration of other “work” a thread performs between transactions is $\mathbf{T_{work}}$

Reader-Writer Lock Usecase (2)

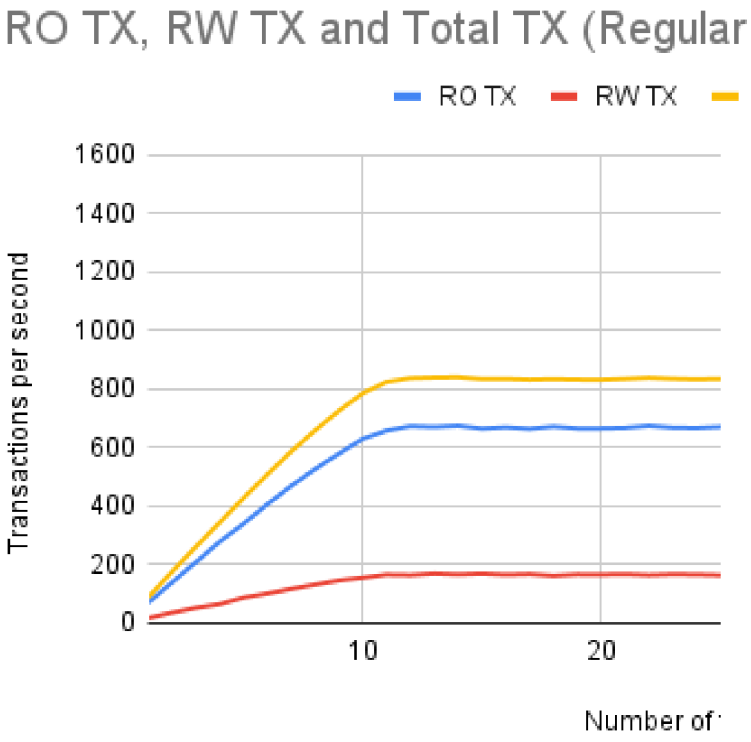
- Implementing the simulation in C++
  - **Database** object that represents the database
  - **Database::read_transaction(int usec)** simulates a read-only transaction: acquire lock as a reader, put calling thread to sleep for “usec” microseconds
  - **Database::write_transaction(int usec)** simulates a read-write transaction: acquire lock as writer, put calling thread to sleep for “usec” microseconds

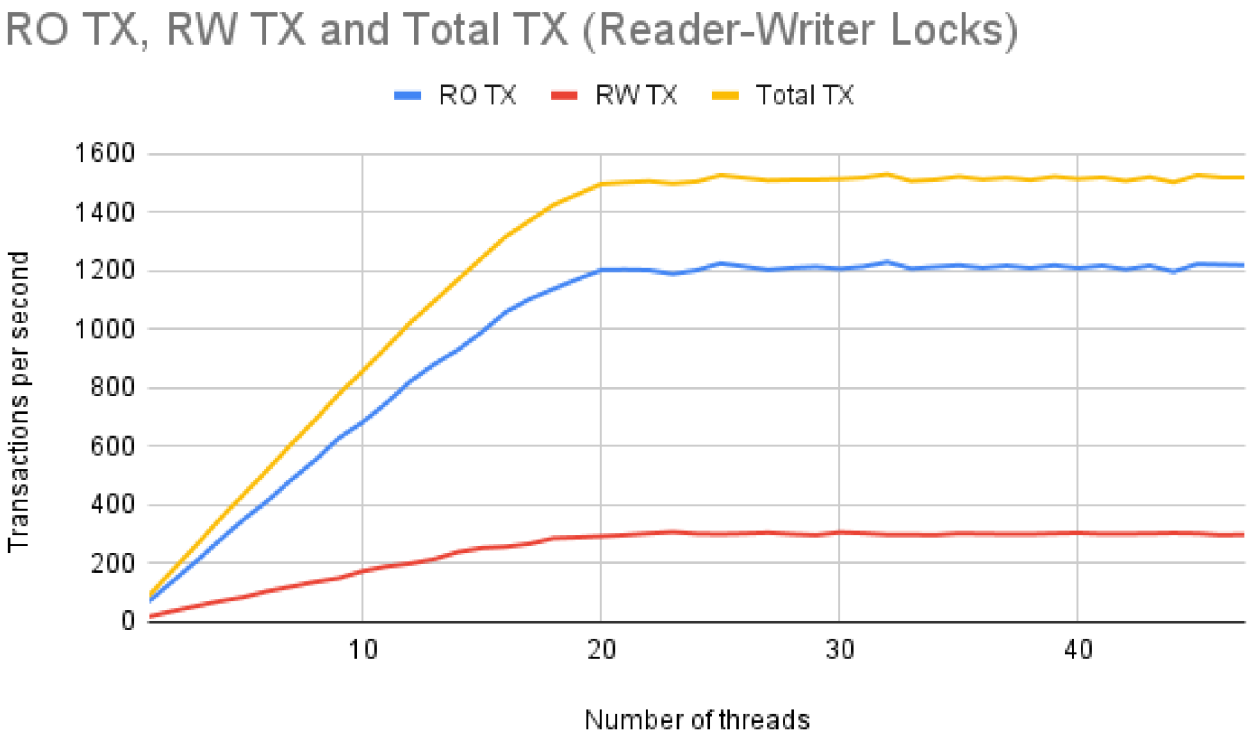
  - Create **N** C++ threads that perform **M** transactions each
  - Use a **PRNG** to determine type of transaction, length of transaction, and duration (work) between transactions

  - Database object also keeps track of **statistics**: number of transactions executed, timestamp of first and last transaction
  - Report the transaction throughput (RO/RW/Total) per second

Reader-Writer Lock Usecase (3)

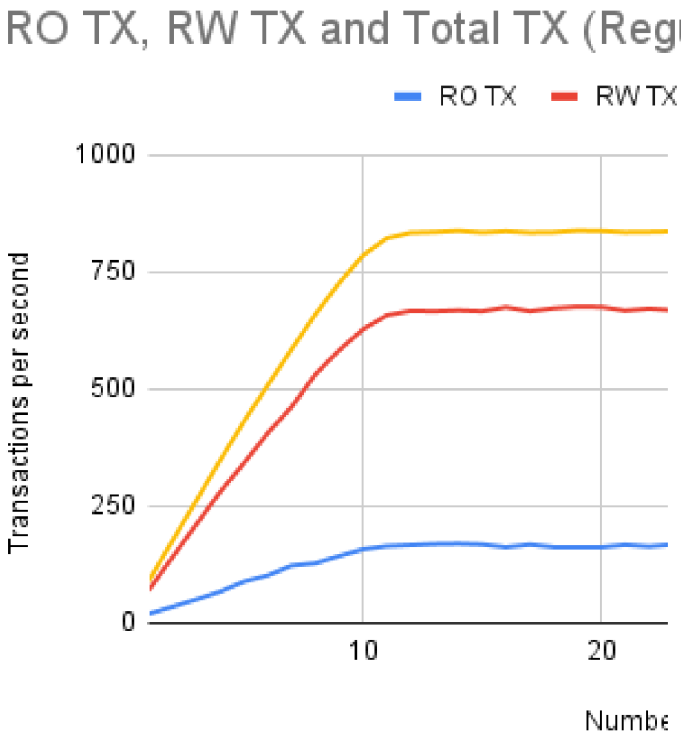
- Example:
  - Do **M=1000** transactions per thread with $\mathbf{T_{work}}$ **= 10000** usec and $\mathbf{T_{write}}$**=**$\mathbf{T_{read}}$**=1000** usec and **P=0.2** (most transactions are reads)
  - Compare implementation using regular locks with RW locks

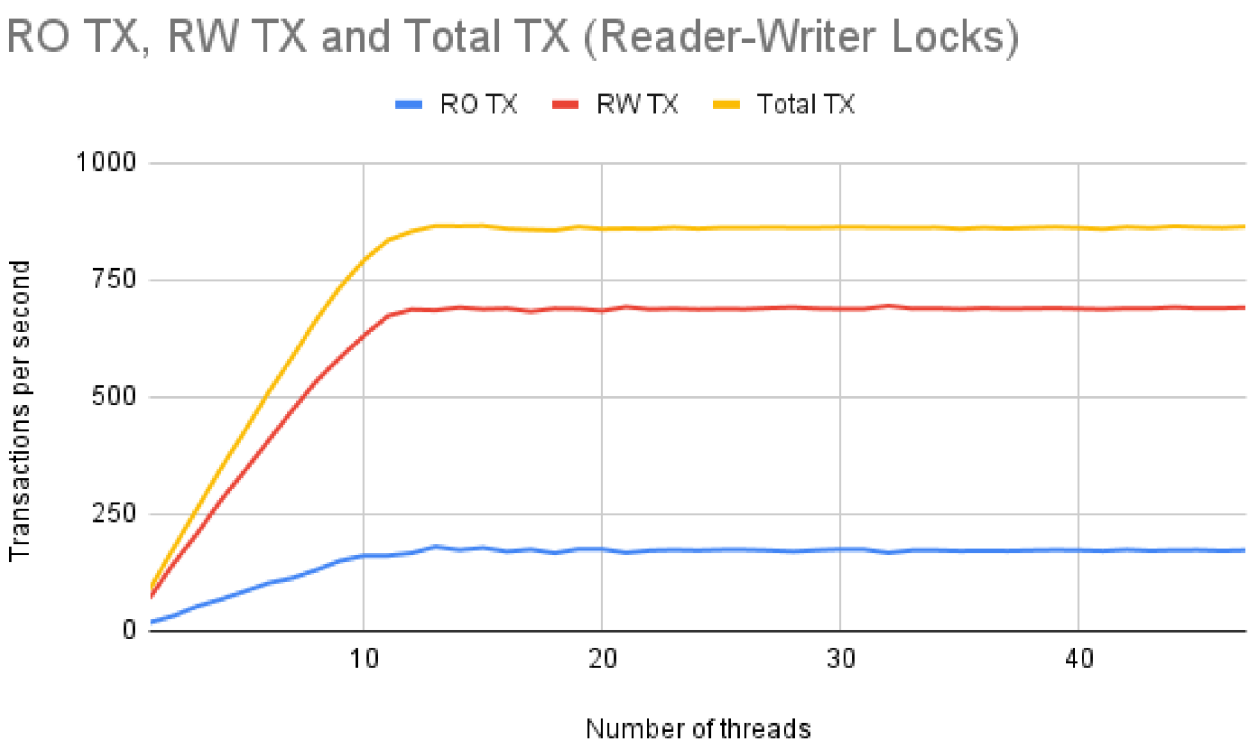




Reader-Writer Lock Usecase (4)

- Example:
  - Do **M=1000** transactions per thread with $\mathbf{T_{work}}$ **= 10000** usec and $\mathbf{T_{write}}$**=**$\mathbf{T_{read}}$**=1000** usec and **P=0.8** (most transactions are writes)
  - Compare implementation using regular locks with RW locks

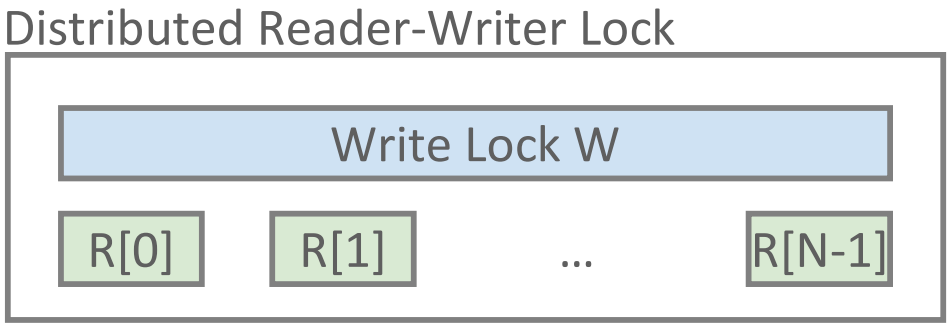




A Distributed Reader-Writer Lock (1)

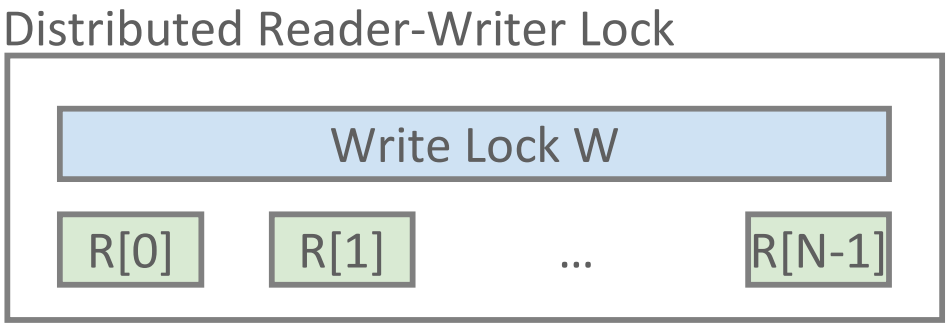
- Useful in a situation where
  - Reading is much more common than writing and locking for reading should be made very cheap at the expense of the rare locking for writing
  - The number of threads is known

- Idea:
  - Each thread has it's own "reader lock"
  - There is a global writer lock



R[i] … Read-Lock of thread i

A Distributed Reader-Writer Lock (2)



R[i] … Read-Lock
of thread I
can be a simple (Peterson) lock

- writer_acquire()
  - acquire the global write lock
  - acquire **all** read locks
- writer_release()
  - release **all** read locks
  - release the global write lock
- reader_acquire()
  - acquire the **local** read lock
- reader_release()
  - release the **local** read lock

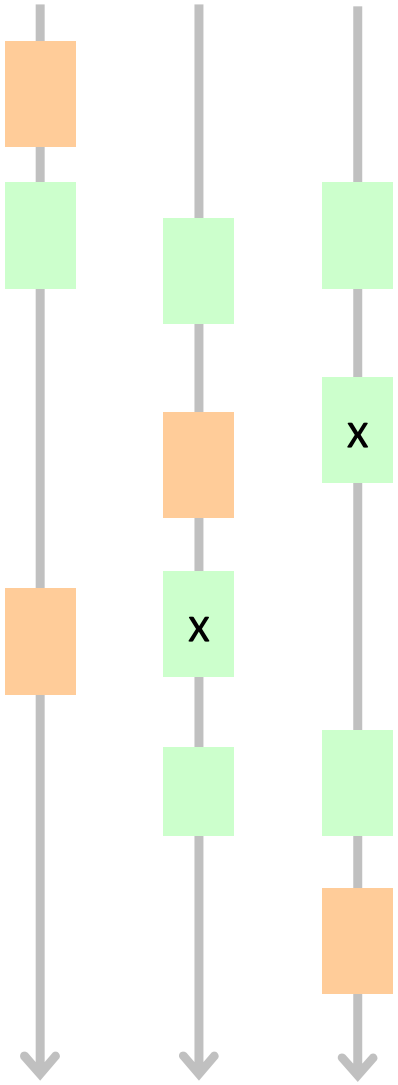
- Q: Reader-preference, Writer-preference or fair?

Sequence Locks (1)

- Problems with the RW locks so far:
  - Readers **need to know** that they are readers, they cannot become writers while executing their read-only critical section
  - Even readers need to **write to the lock metadata** during their reader_acquire() call => performance implication

- Sequence Locks
  - Mutual exclusion is provided **for writers** only (just like with regular locks)
  - Readers can **speculatively** execute their read-only critical section
  - Afterwards they can find out if their execution overlapped with an active writer and if so, they have to **retry**
  - Promotion to writer status is also offered

Sequence Locks (2)



R/W critical section (CS), the sequence lock provides mutual exclusion, as usual

Attempted and successful read-only CS

x Attempted and unsuccessful read-only CS that needs to be retried

Sequence Lock Pseudocode (1)

```
class SeqLock {
  int n = 0;      // even: read-only phase
                  //  odd: held by a writer
}
```

- Write acquire/release behaves like a TTAS lock:

```
void SeqLock::writer_acquire() {
  int seq;
  do {
    seq = n.load();
  } while( seq%2==1 ||              // spin until seq is even
           !CAS(&n, seq, seq+1) ) // and CAS is successful
  fence(R||RW);
}
```

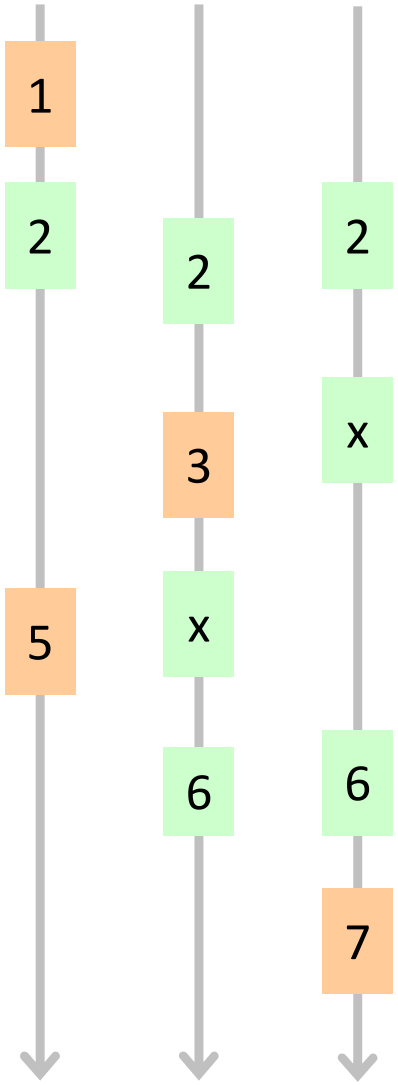
```
void SeqLock::writer_release() {
  int seq = n.load();     // seq will be odd
  n.store(seq+1, RW||);  // store next even number -> released
}
```

Sequence Lock Pseudocode (2)

- Readers start their “transaction” and can then verify if it was interruped by a writer or not

```
int SeqLock::reader_start() {
  int seq;
  do {
    seq = n.load();
  } while( seq%2==1 ) // spin until seq is even (no writer active)
  fence(R||R)
  return seq;
}
```

```
bool SeqLock::reader_validate(int seq) {
  return n.load(R||)==seq;  // still the same seq. number?
}
```



R/W critical section (CS), the sequence lock provides mutual exclusion, as usual

Attempted and successful read-only CS

x Attempted and unsuccessful read-only CS that needs to be retried

Sequence Lock Pseudocode (3)

- Bonus: One of the readers can be promoted to writer status
  - Any reader must be prepared to be interrupted by a concurrent writer anyway

```
bool SeqLock::become_writer(int seq) {
  if( CAS(&n, seq, seq+1, R||) ) {
    fence(R||W);
    return true;
  }
  return false;
}
```

Reader-Writer Locks in C++

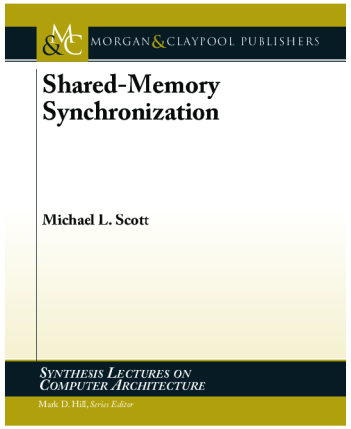
- std::shared_mutex offers the equivalent of R-W locks
  - Terminology is more general, though:
  - There are two classes of ownership: **exclusive** and **shared**
- Exclusive ownership
  - lock() / unlock(), try_lock() functions
- Shared ownership
  - lock_shared() / unlock_shared() / try_lock_shared() functions
- There is also a std::shared_timed_mutex

Summary

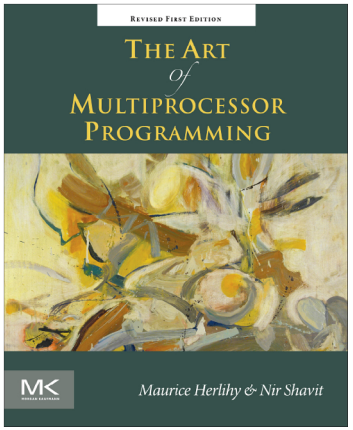
- We have seen many special-case adaptations
  - Support for **asymmetric** access patterns (preferred threads)
  - Support for **locality** with cohort locks
  - Support for **reader-writer** scenarios with various tradeoffs

- We have also seen that locks are not always the ideal solution
  - **Lock-free data structures** may offer better performance and fault tolerance (cf. future lecture)
  - Locks are conservative, sometimes the protection they offer may not actually be needed if the interleaving of threads is such that no conflicts arise
  - In this case a **speculative approach** may be better: Just execute all critical sections hoping that they don't overlap (there is no conflict)
  - If there is a conflict, roll-back and retry
  - This is called **transactional memory** (cf. future lecture) and sequence locks may be seen as a special case

Further Reading



- Michael L. Scott *"Shared-memory synchronization"* Synthesis Lectures on Computer Architecture (2013)
  - Chapter 4 “Practical Spin Locks”
  - Chapter 6 “Read-mostly Atomicity”



- Maurice Herlihy, Nir Shavit *"The Art of Multiprocessor Programming"*
  - Chapter 7 “Spin Locks and Contention”